API Gateway Security Report

Prepared by: Charmaine Mangorima
Date: January 15, 2026
Target Application: Test API hosted on Render
Testing Type: API Gateway Security Implementation

1. Executive Summary

This report documents the implementation of an API Gateway security layer for an API. The goal of the lab was to secure a backend API using a cloud provider while enforcing strong authentication, HTTPS-only communication, rate-limiting and access control.

Amazon API Gateway combined with Amazon Cognito was used to implement OAuth2/OpenID Connect authentication in front of the existing API hosted on Render. This approach effectively mitigated security weaknesses such as unauthenticated access and unrestricted resource consumption.

2. Scope

In-Scope

- The test API and all its endpoints
- Authentication and authorization at the API Gateway layer
- HTTPS enforcement
- Rate limiting and abuse prevention

Out-of-Scope

- Backend application source code
- Database and storage security
- Frontend user interface

3. Methodology

Steps

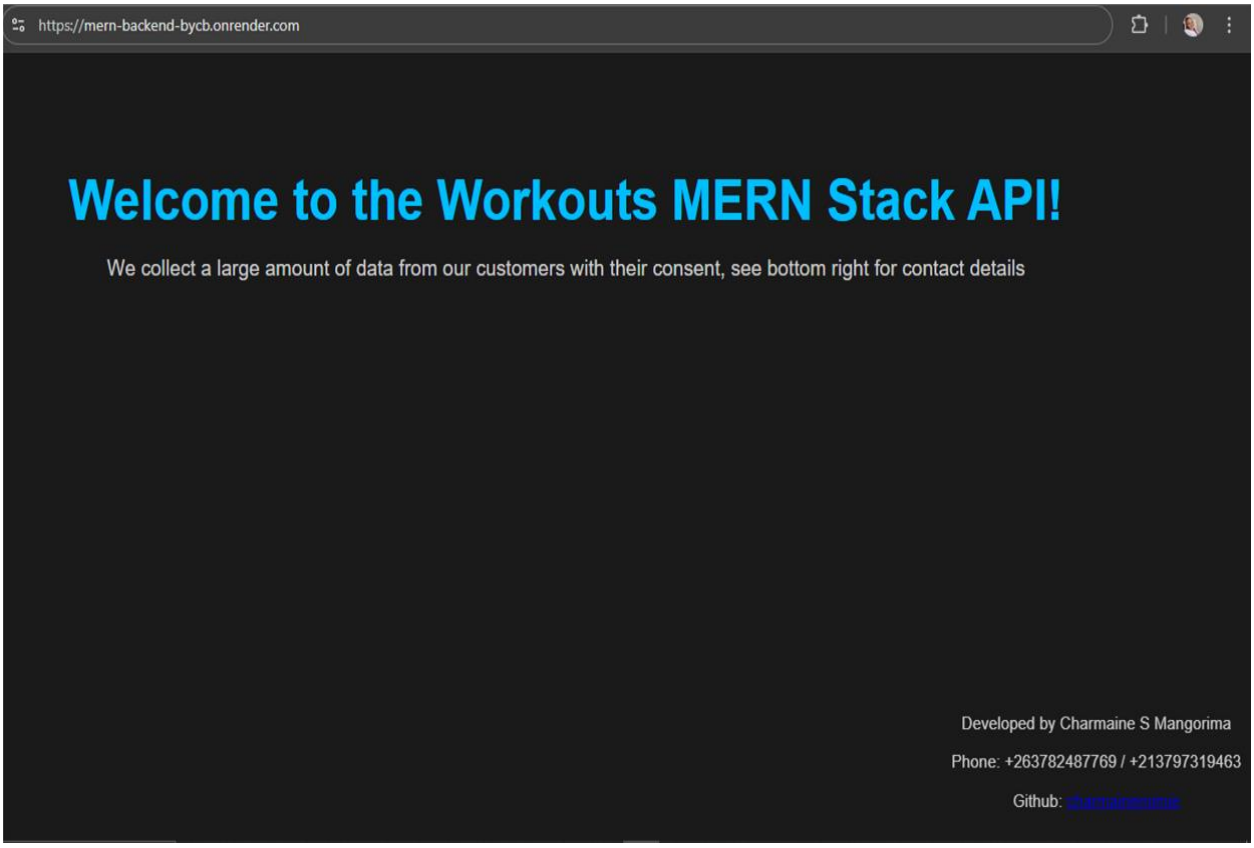
1. Deployed AWS API Gateway in front of the Render-hosted backend API
2. Implemented OAuth2/OpenID Connect authentication using AWS Cognito
3. Enforced HTTPS-only communication (by default)
4. Configured rate limiting rules
5. Tested authenticated and unauthenticated access

Tools Used

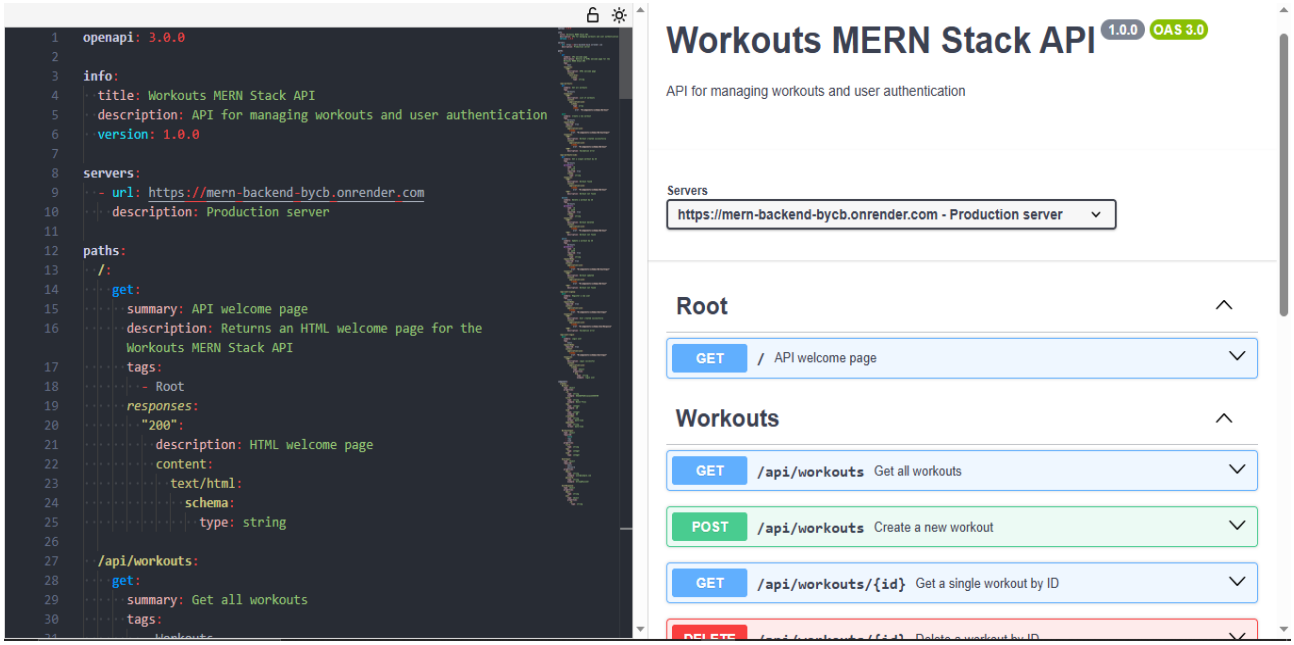
- AWS API Gateway
- AWS Cognito
- Postman
- jwt.io and xjwt.io
- Swagger Editor

4. Implementation

A simple test API was used (**https://mern-backend-bycb.onrender.com/**).

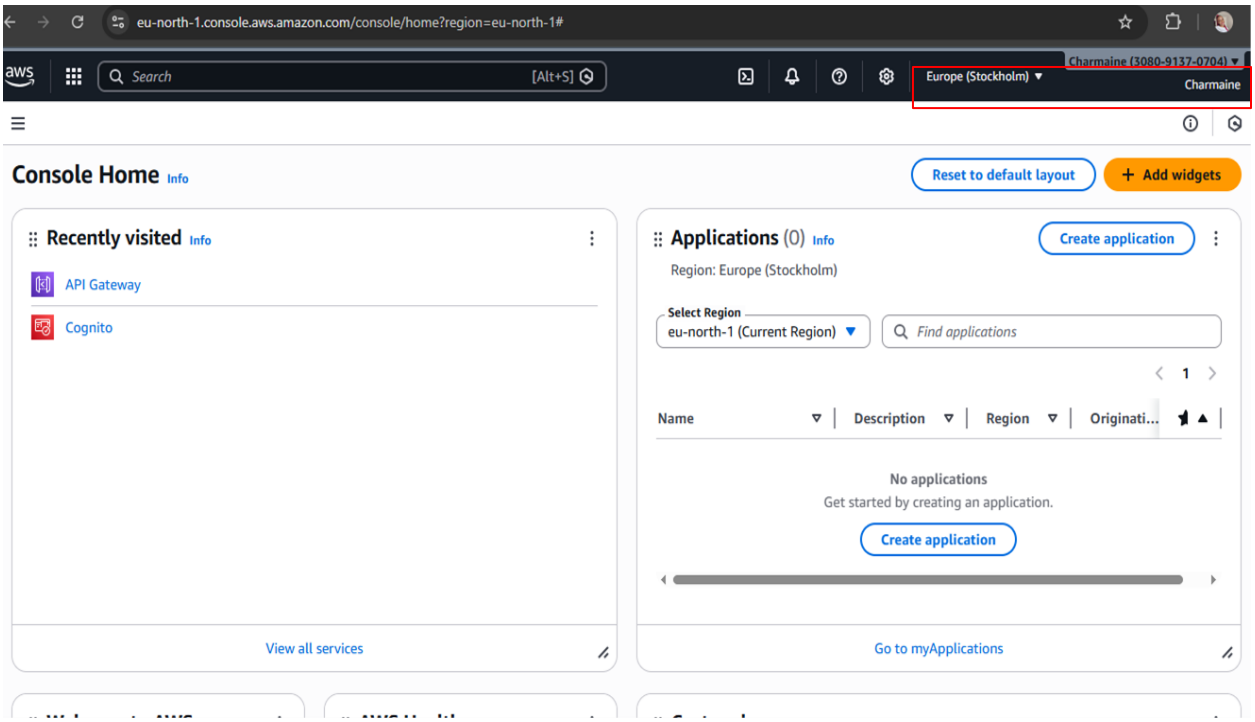


I used swagger editor to get a clear view of the API endpoints



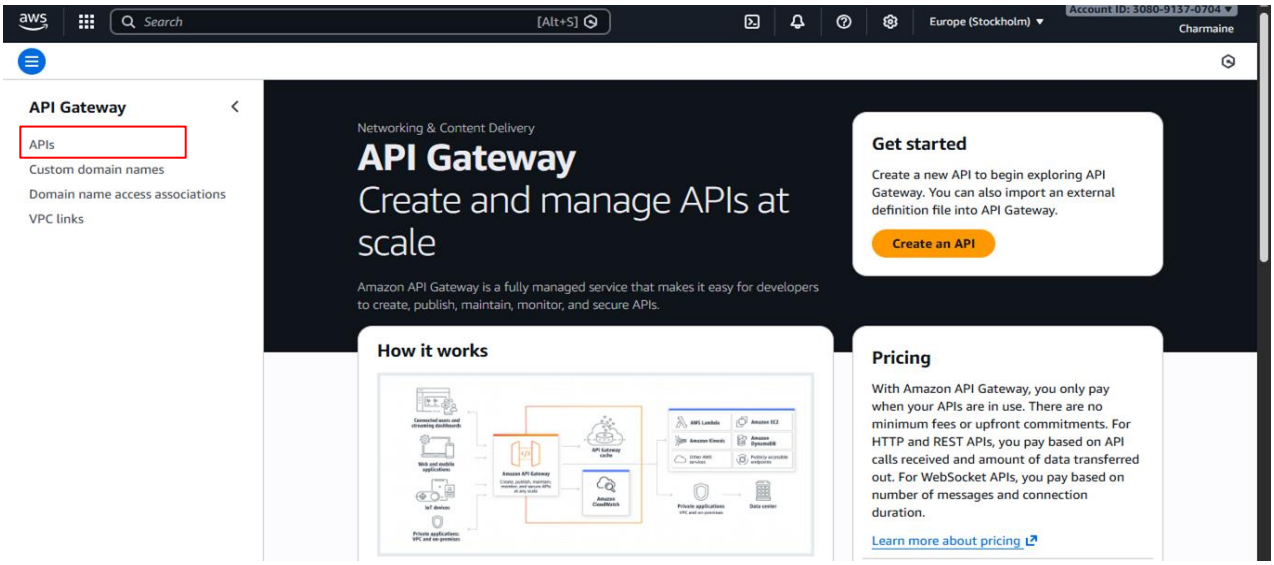
STEP 1: Creating an AWS account and logging in as root user.

I went to the AWS console and signed up for a free account as shown by the Console Home below

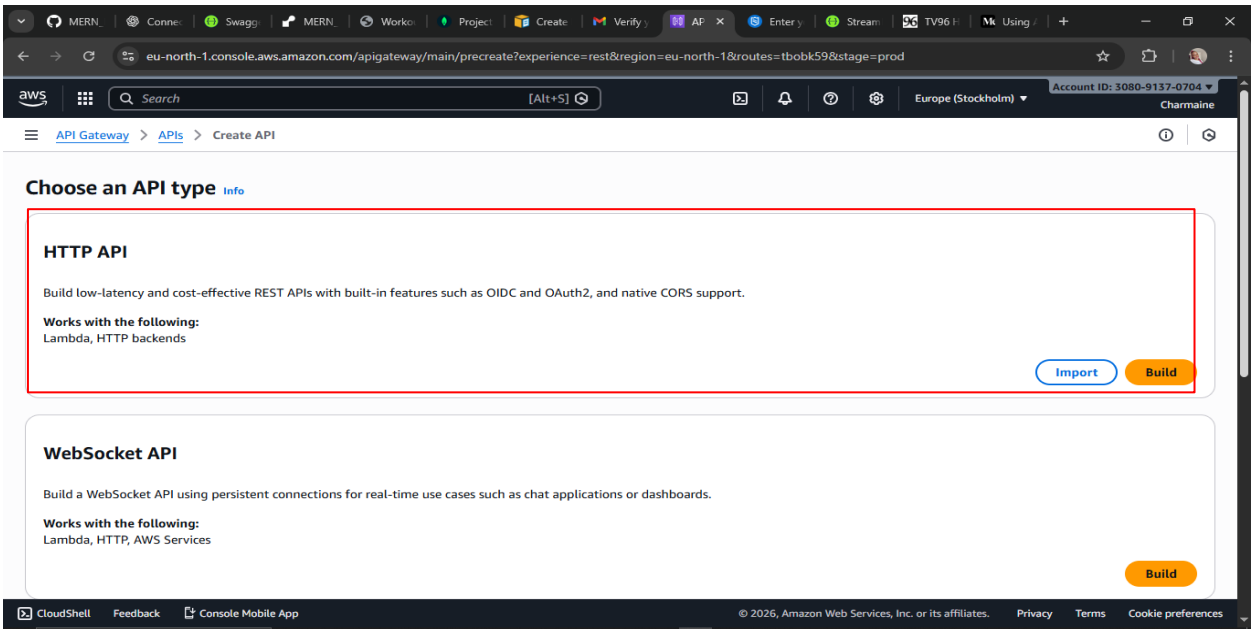


STEP 2: Creating an API Gateway

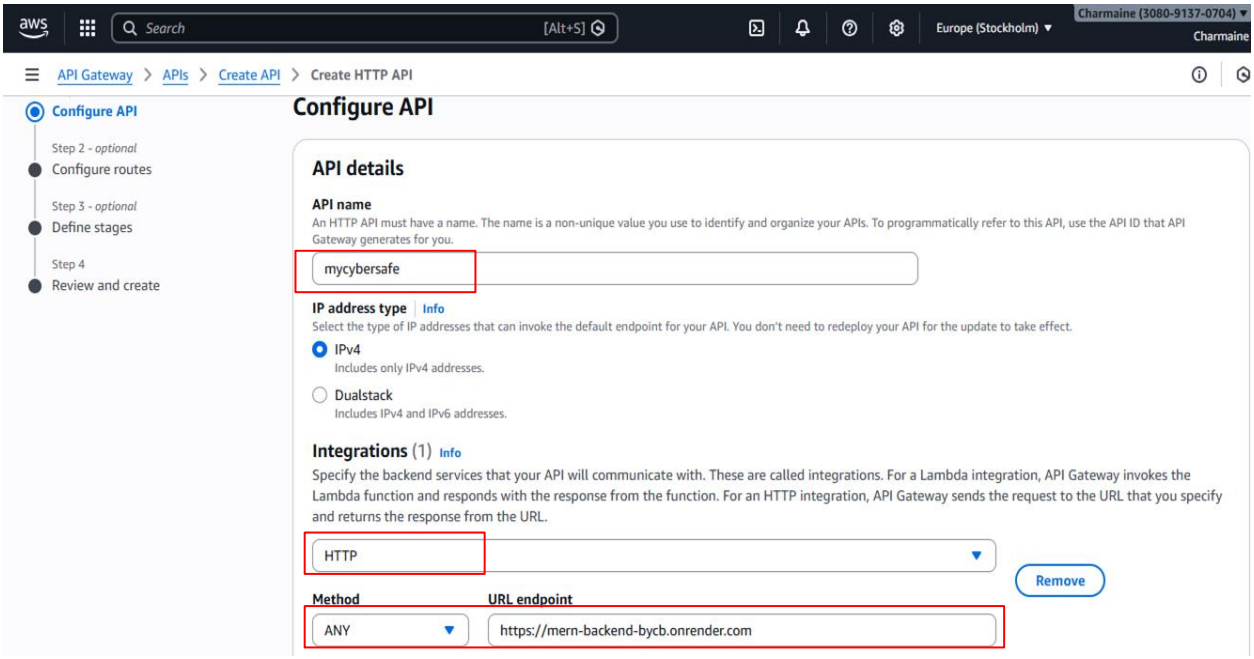
On the Console, I went to **AWS API Gateway,** under APIs, I chose **Create API**



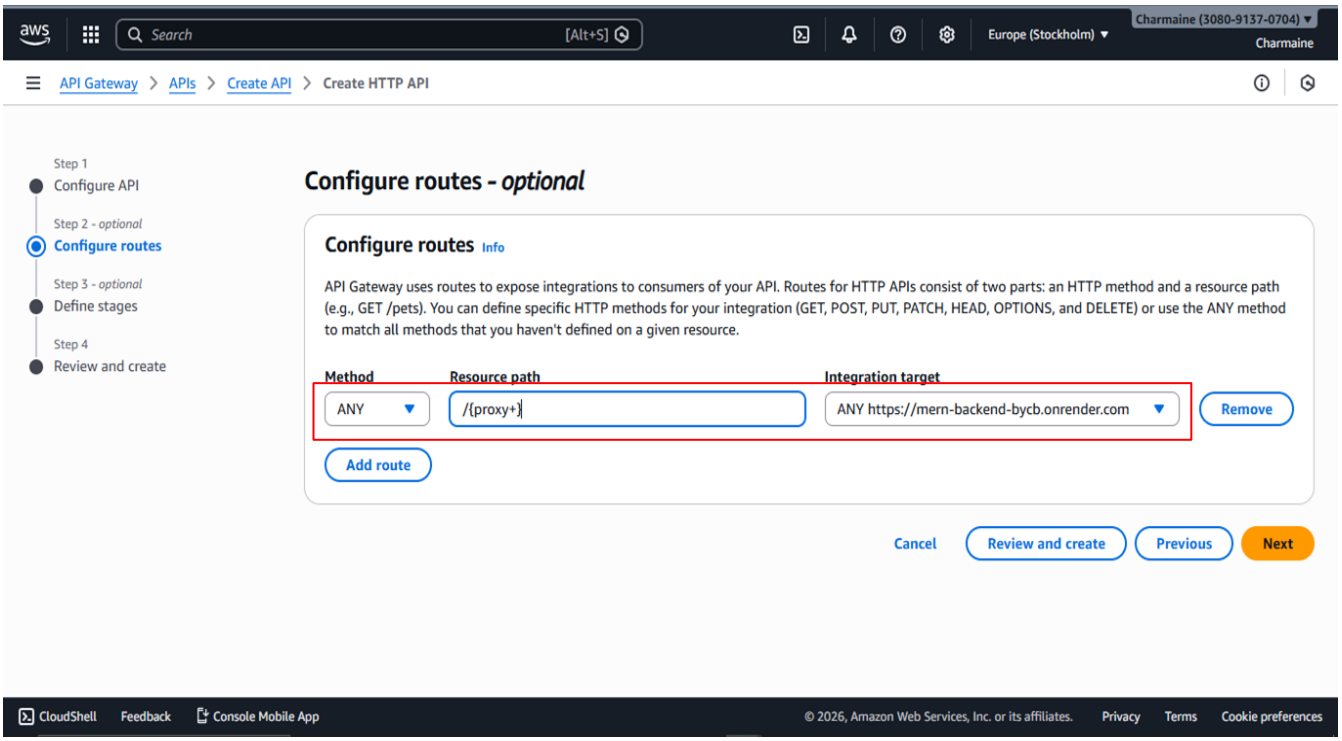
For API type I chose **HTTP API** then **Build** (it's simpler and cheaper)



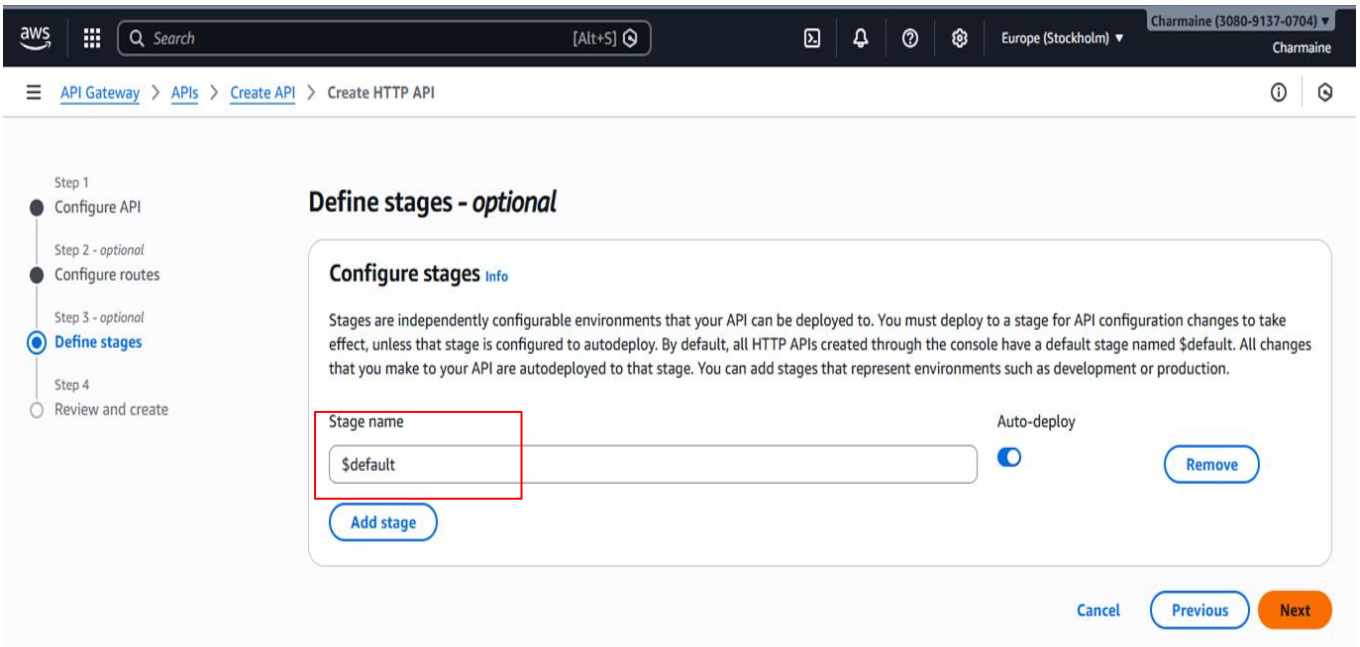
I configured the API by providing the API name **(mycybersafe)** and I added my Render backend URL with the ANY method which catches all request types to the API.



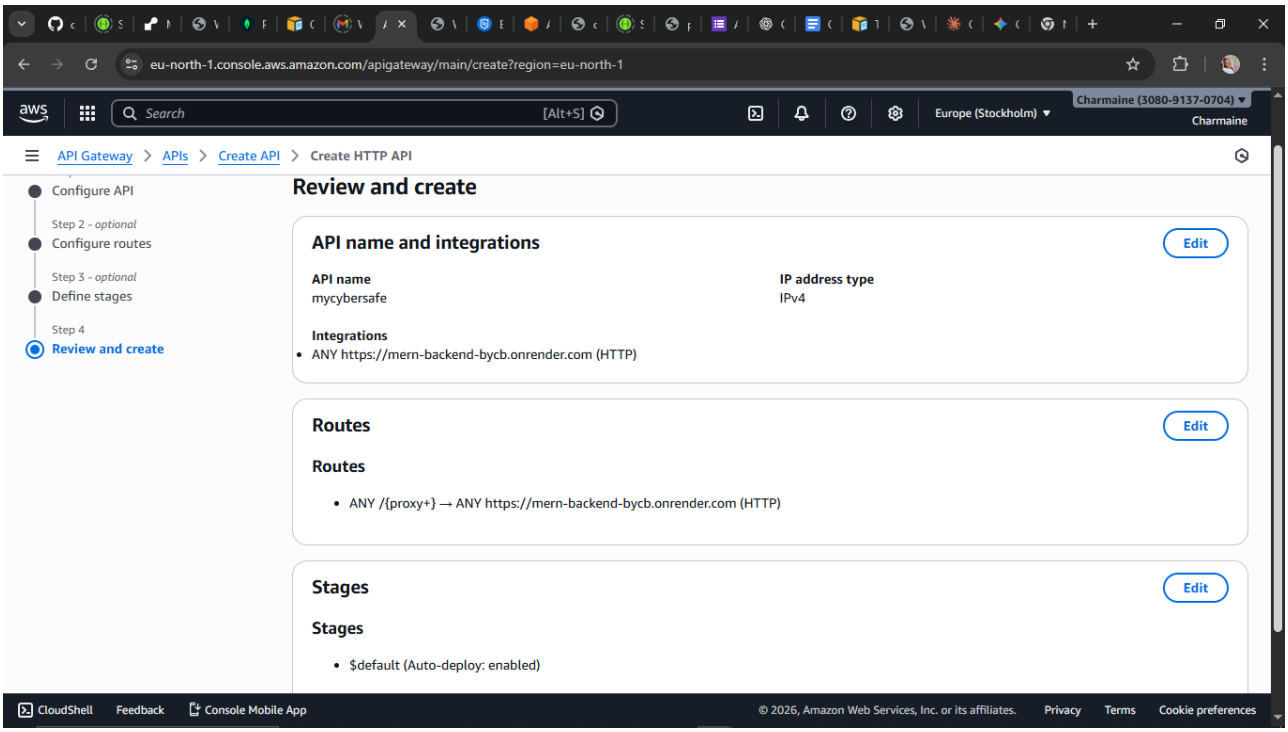
For route configuration, I added the route: ***ANY /{proxy+}*** (this catches all paths) and associated it with the previously created integration

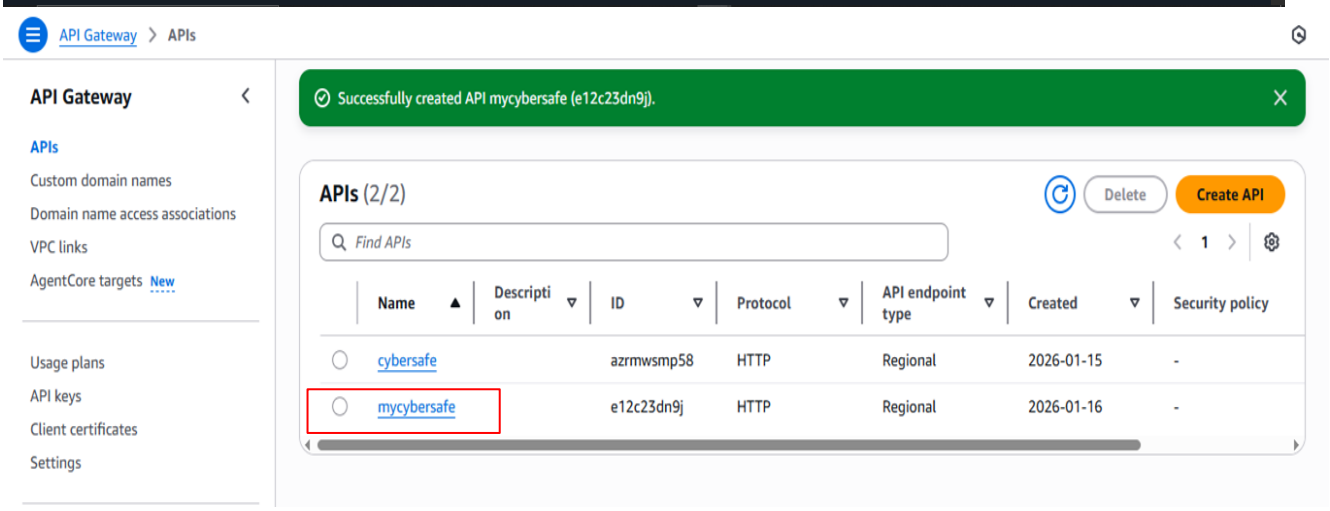


For stages, I went with the default stage name

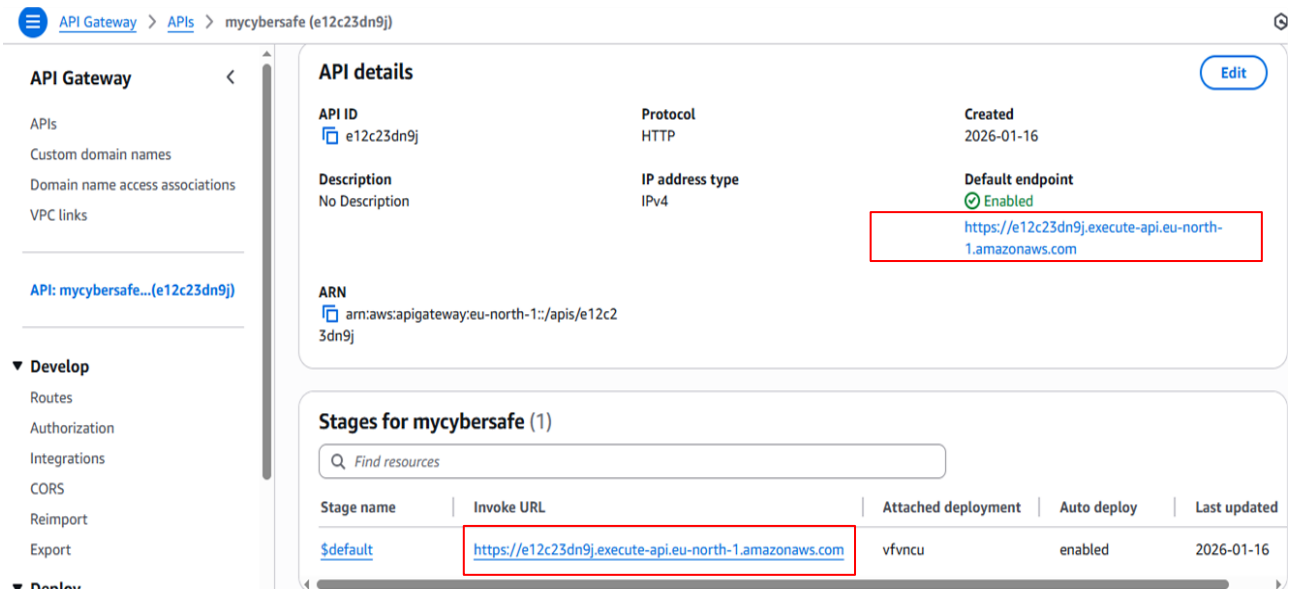


I reviewed and created my API and it is now shown under APIs

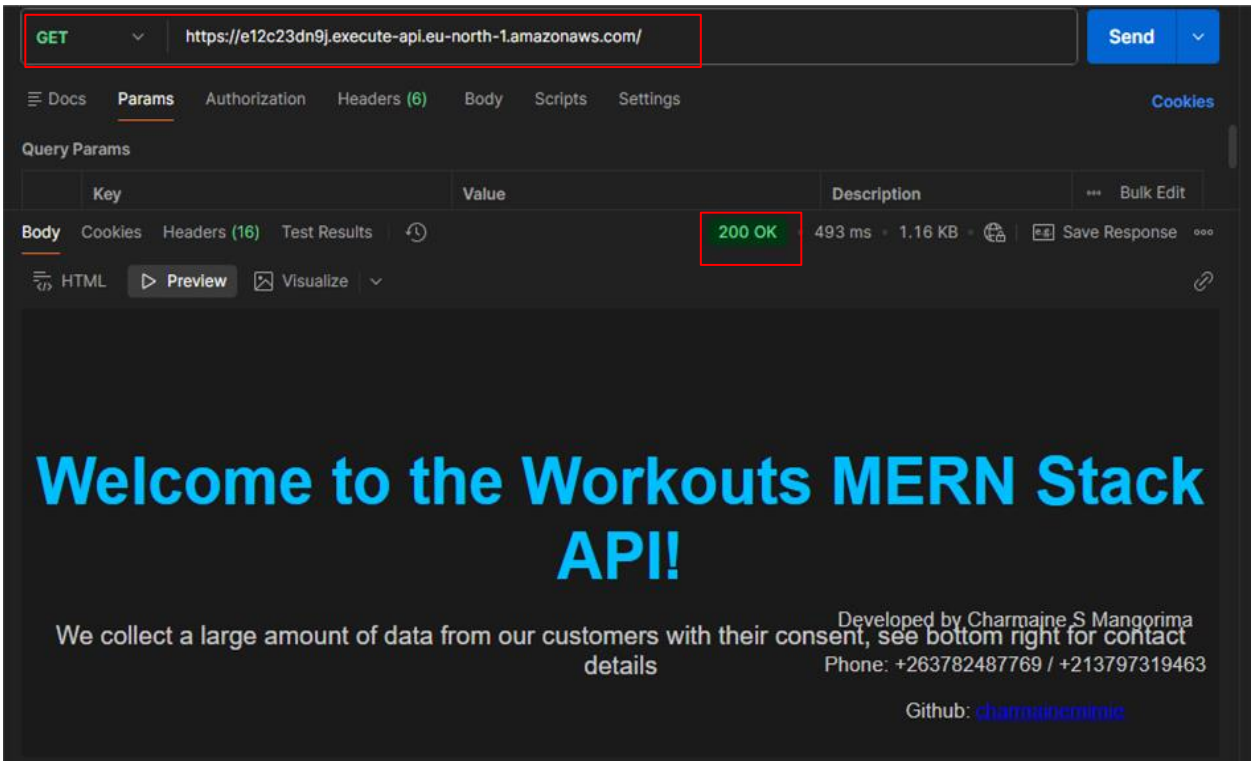




I verified if my API is now behind the AWS API gateway by sending a GET request the URL https://e12c23dn9j.execute-api.eu-north-1.amazonaws.com/

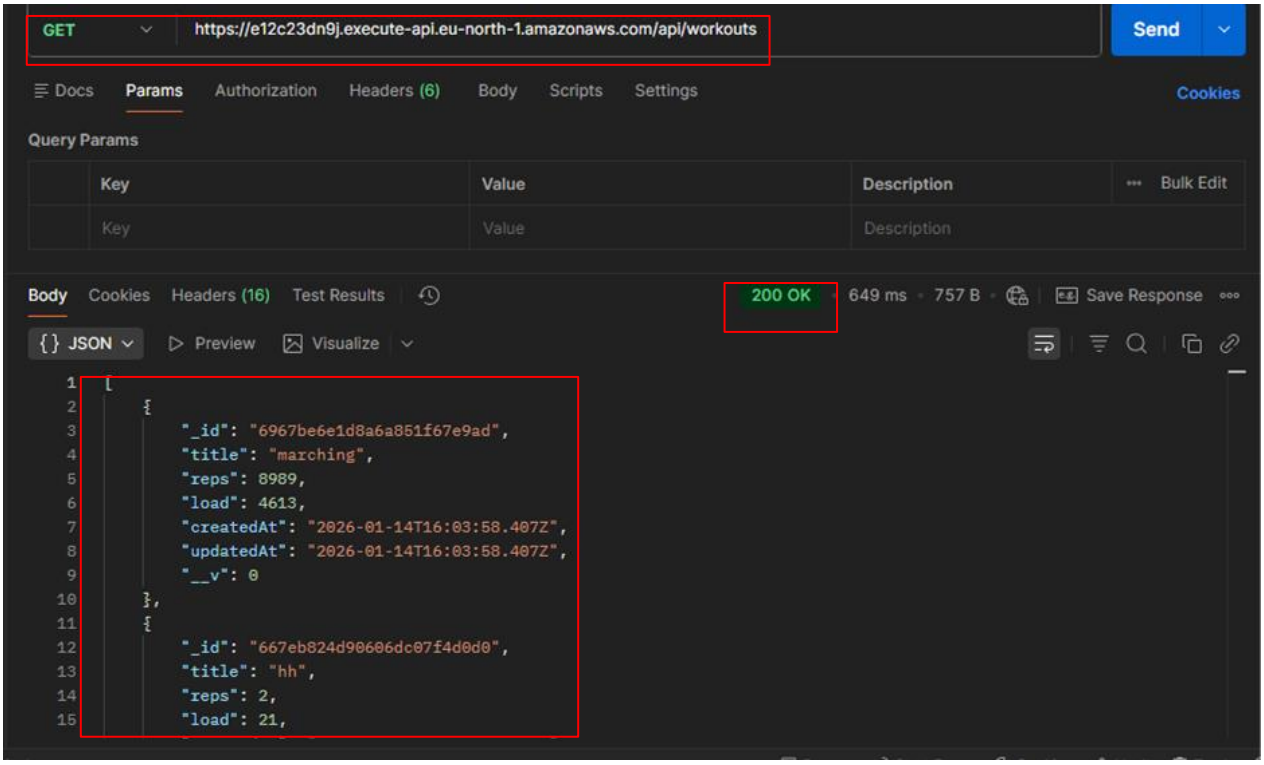


In Postman, I sent a GET request to the URL and the response was the API Home page . Now my API was behind the AWS API Gateway.



I noticed that all my requests to the endpoints via the API Gateway were routing to the home. I edited the integration to https://mern-backend-bycb.onrender.com/{proxy} so that AWS knows to pass the {proxy} variable forward.

In Postman, using the API Gateway URL, I sent a GET request to the ***/api/workouts*** endpoint which is accepted by the API.

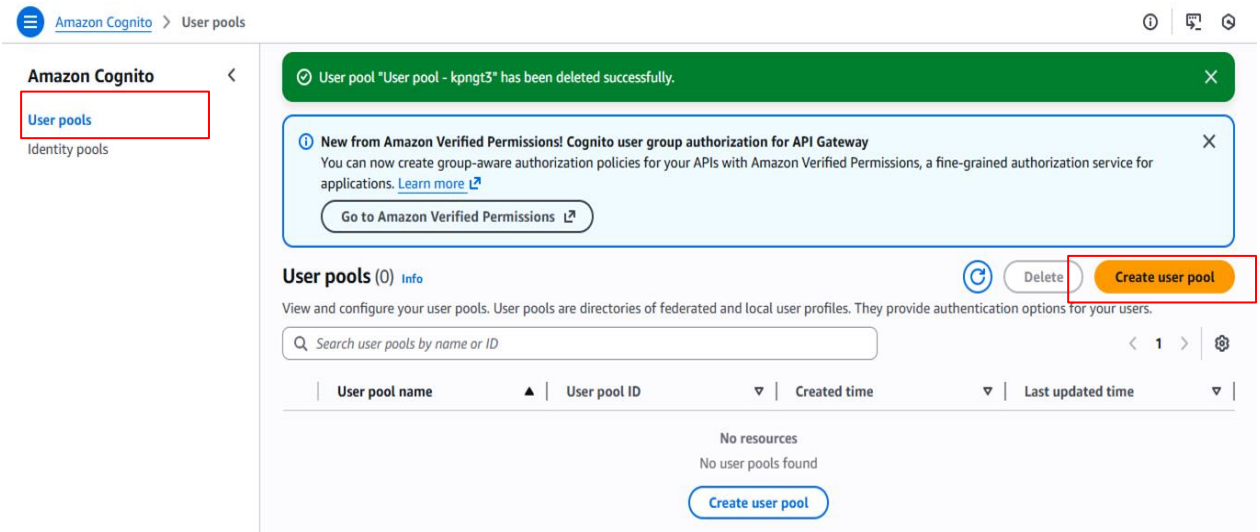


Step 3: Implementing OAuth2 by using Amazon Cognito

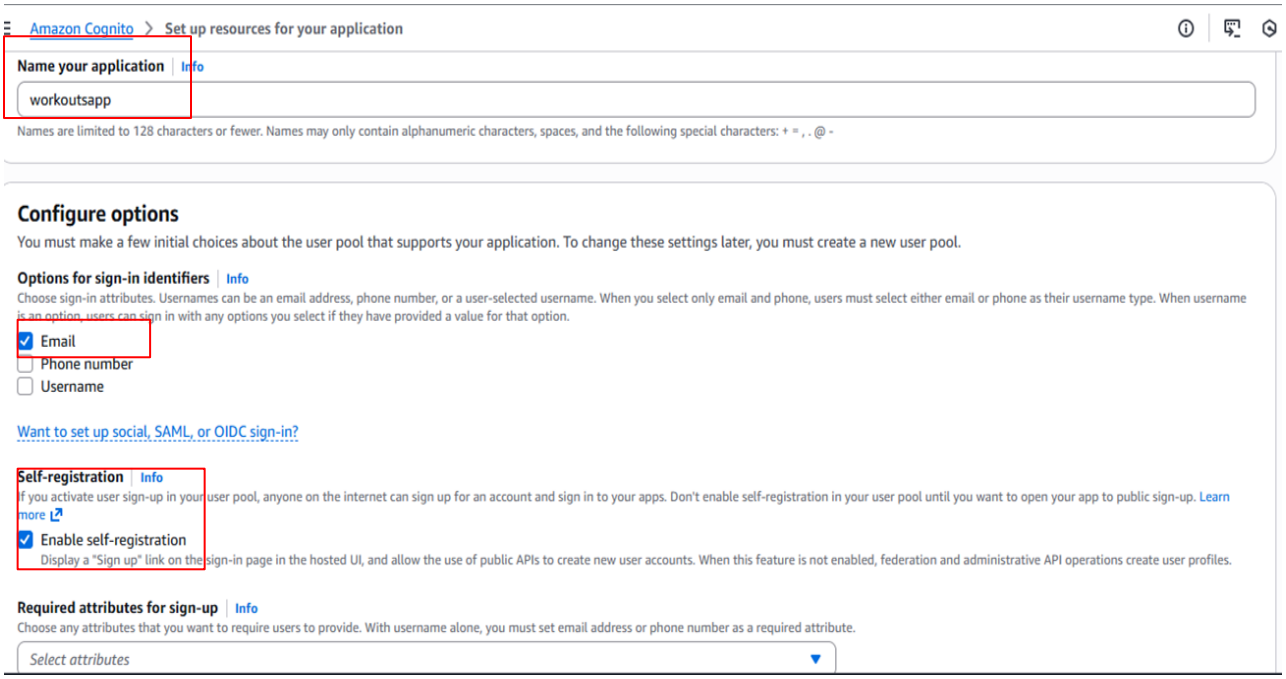
OAuth 2.0 serves as the authorization framework. It seperates authentication from the application logic by using Amazon Cognito to issue signed JWT Access Tokens. These tokens contain specific Scopes and Identity Claims that allow the API Gateway to enforce security policies and data isolation without the backend ever handling sensitive user credentials.

Now that the API was behind the API Gateway, I implemented OAuth2 using **Amazon Cognito**. Since the API was public facing and does not have its own JWT logic, AWS acts as the guard that handles the login and then passes valid requests to the API.

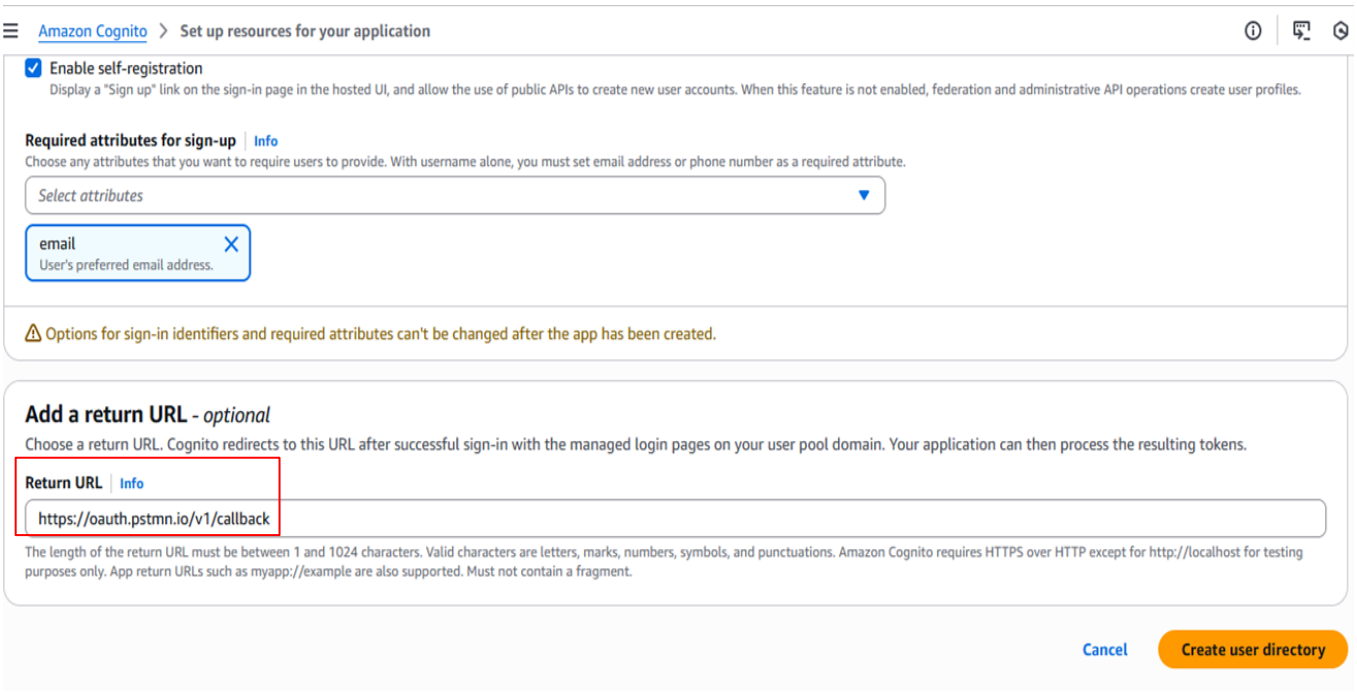
On the Console Home I went to **Amazon Cognito**, under **User Pools** I clicked **Create User Pool.**



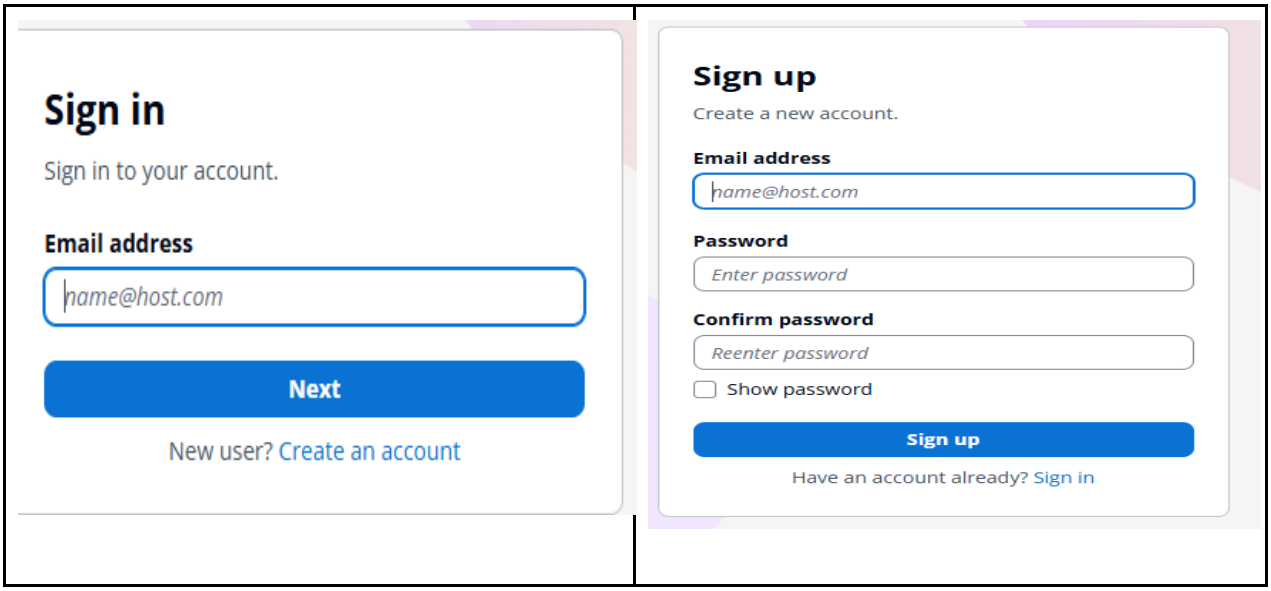
I went ahead and setup the User pool



For the return URL I used a Postman callback URL(https://oauth.pstmn.io/v1/callback) so that it would be easier for me to test in Postman then I created the User Directory.



Amazon Cognito then provides a built in authentication UI.



Step 4: Applying Security Controls

4.1 Authentication (OAuth2/JWT):

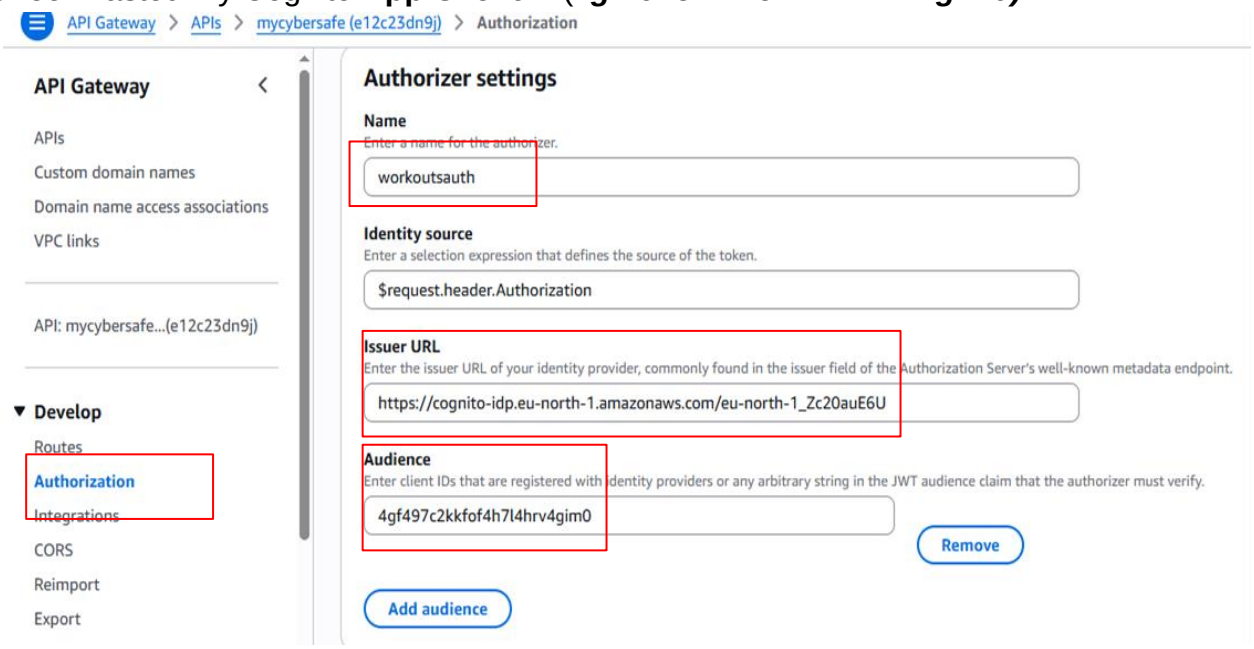
Description

OAuth 2.0 provides a defense in depth layer by decoupling authentication from the application logic. It protects the API by enforcing signed, time-limited tokens (JWTs) that carry specific permissions (Scopes). This ensures that even if a request is intercepted, the window of risk is small, and the user's actual credentials remain safely stored within the AWS Cognito infrastructure.

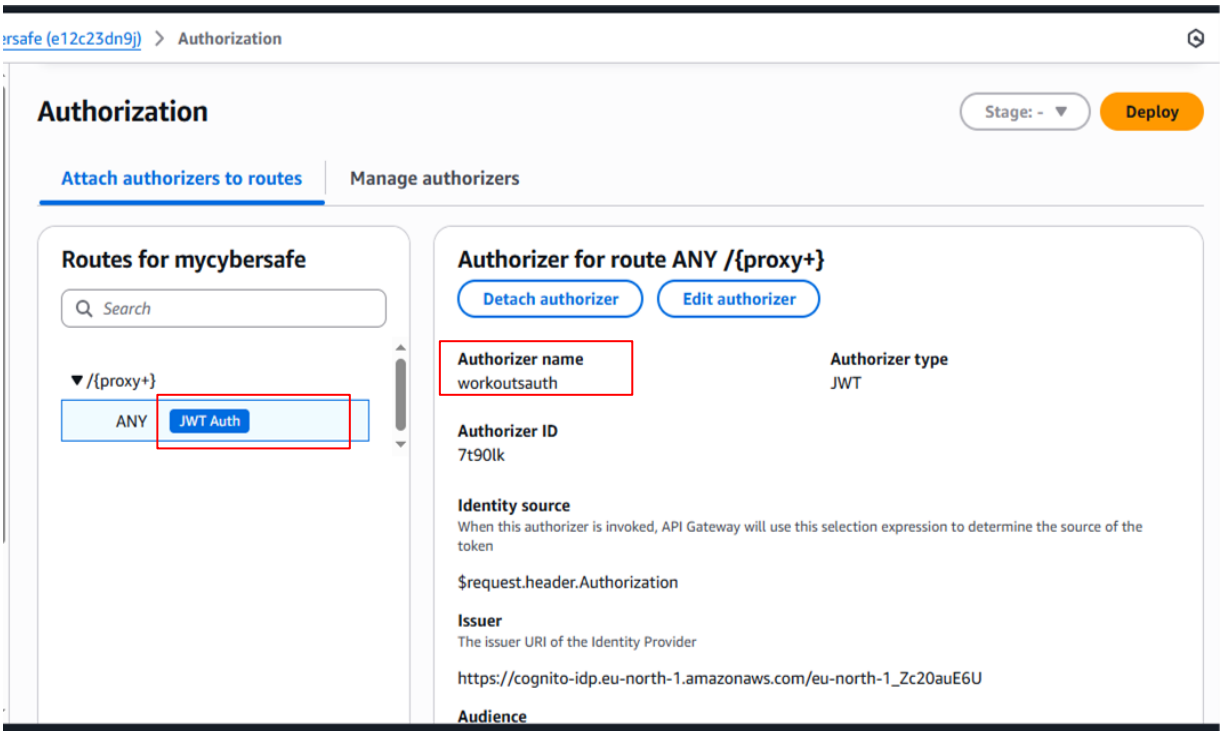
1. User authenticates with Cognito (OAuth provider)
2. Cognito issues JWT access token (OAuth 2.0)
3. Client includes token in Authorization header
4. API Gateway validates JWT signature and claims
5. Valid requests forwarded to backend
6. Invalid/missing tokens receive 401 Unauthorized

Implementation

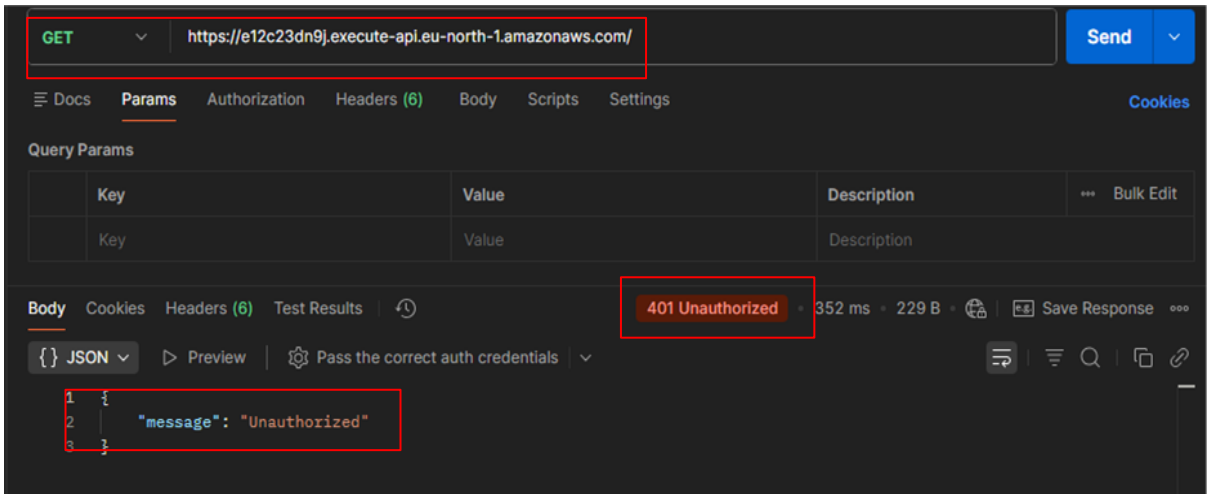
In the API Gateway console, I went to Authorization and created a new JWT authorizer for the ***mycybersafe*** API.
Issuer URL: Copied this from my Cognito User Pool (***https://cognito-idp.eu-north-1.amazonaws.com/eu-north-1_Zc20auE6U***)
Audience: Pasted my Cognito **App Client ID**(***4gf497c2kkfof4h7l4hrv4gim0)***



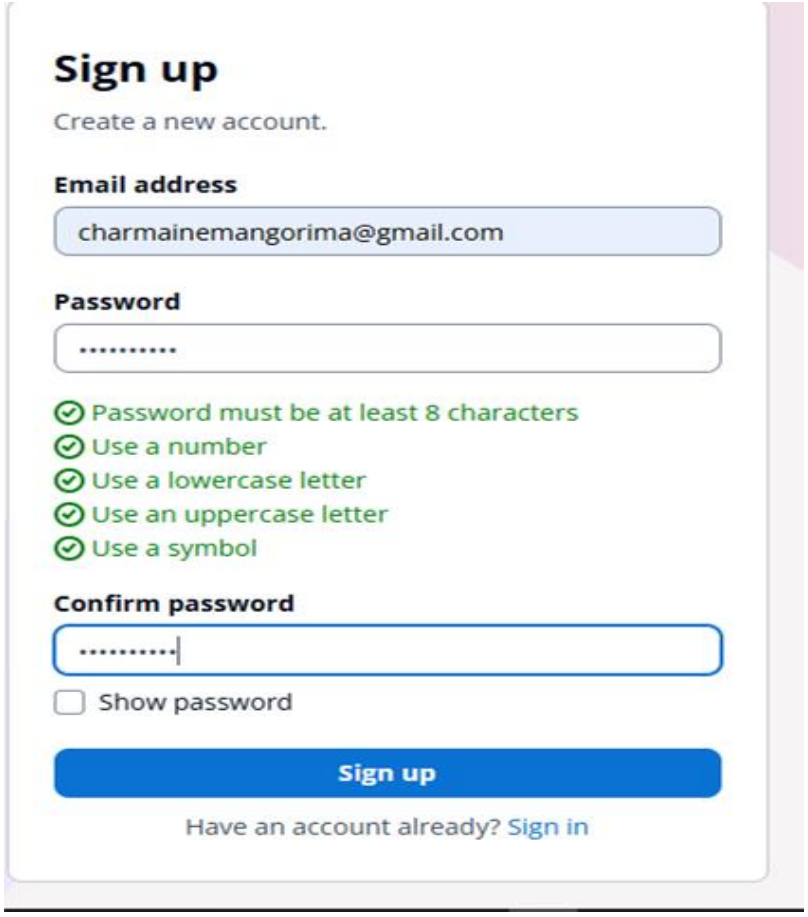
I attached this authorizer to my API routes



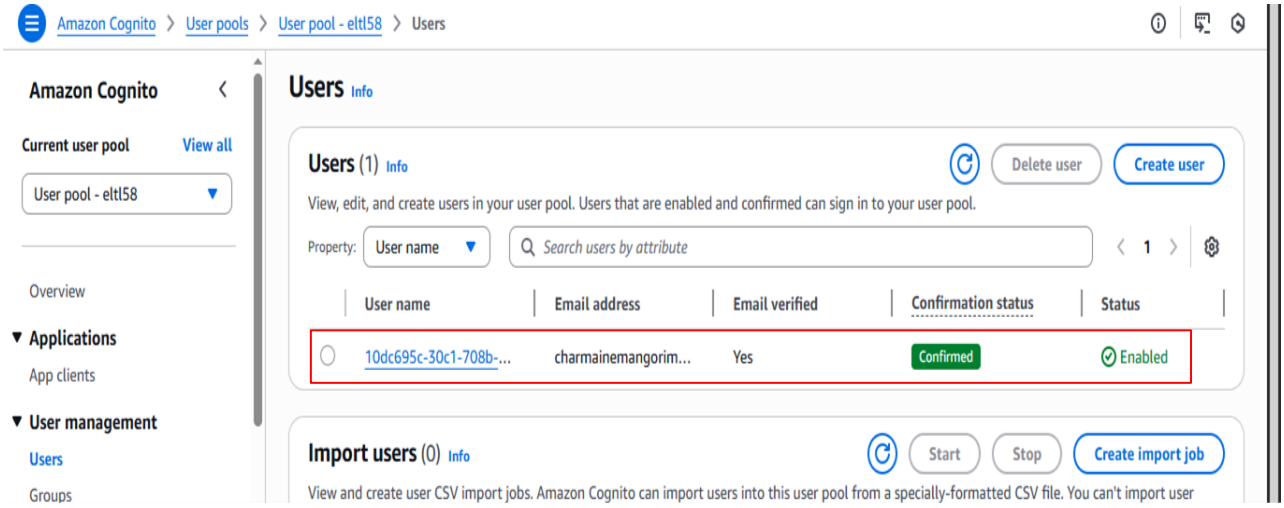
In Postman I tested to see if the API is indeed protected.
I sent a GET request to the API Gateway URL and the response was an error 401(Unauthorized) because I was not authenticated



Using the Amazon Cognito Signup UI, I created a test user account using my email and I verified using the code sent to the email



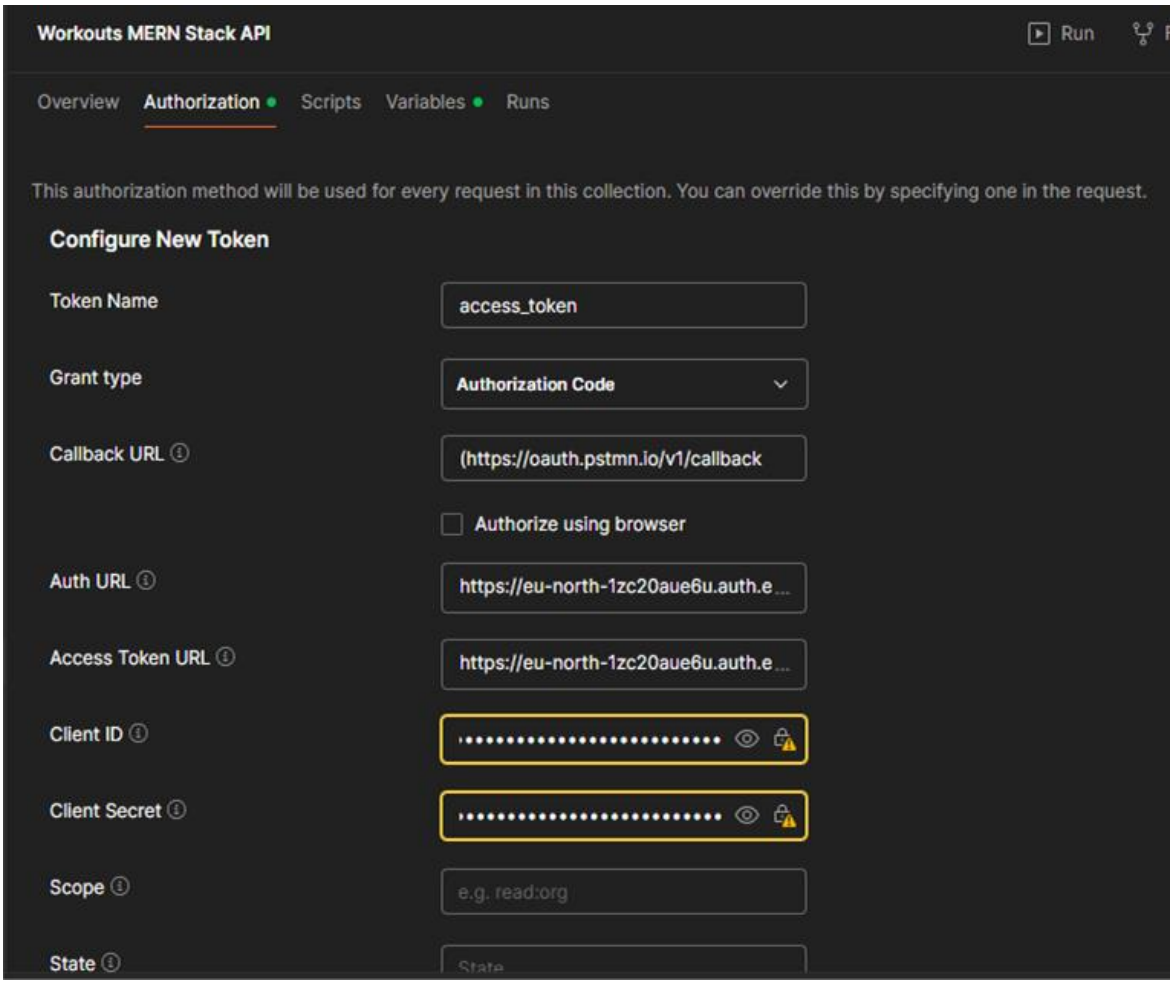
The user now appears under Users in Cognito



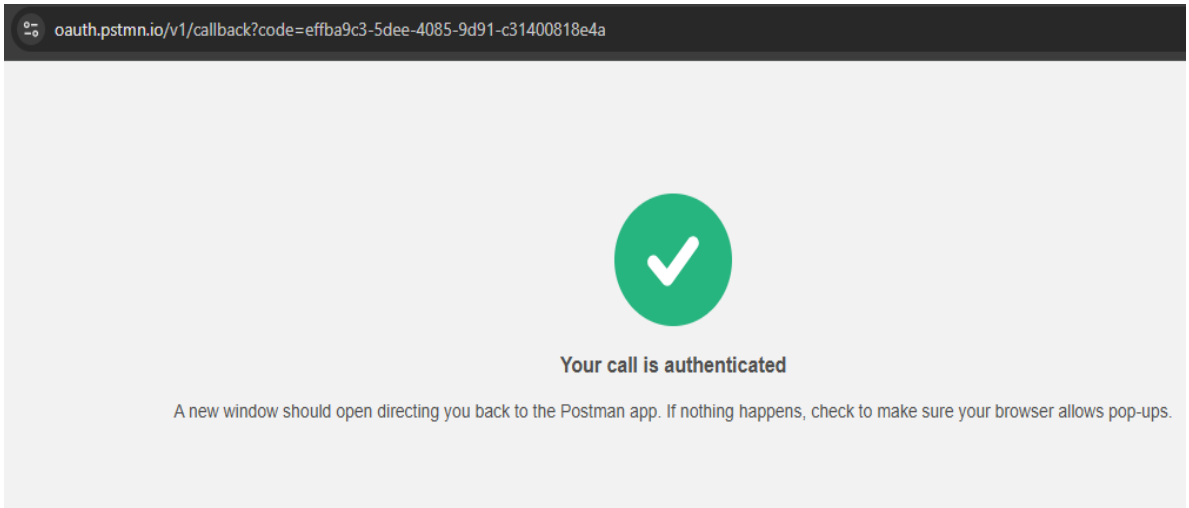
In Postman, I clicked on the **Workouts MERN Stack API** Collection, then under authorization I configured the following and selected **Get New Access Token**:

Authorization type is Oauth2.

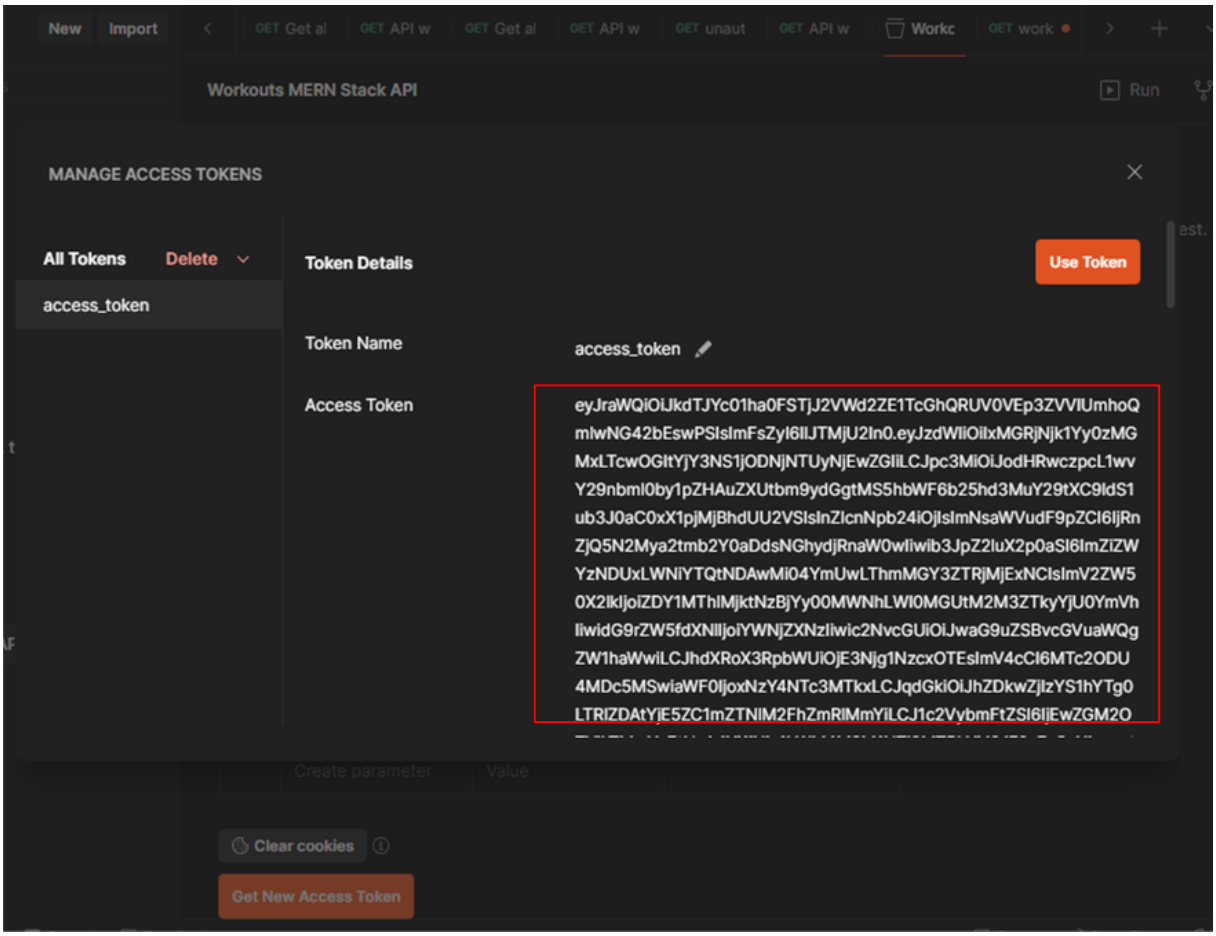
For the for the **Auth URL** and **Access Token URL** I copied the URL from the AUTH UI in the browser and added **/oauth2/authorize** and **/oauth2/token** respectively



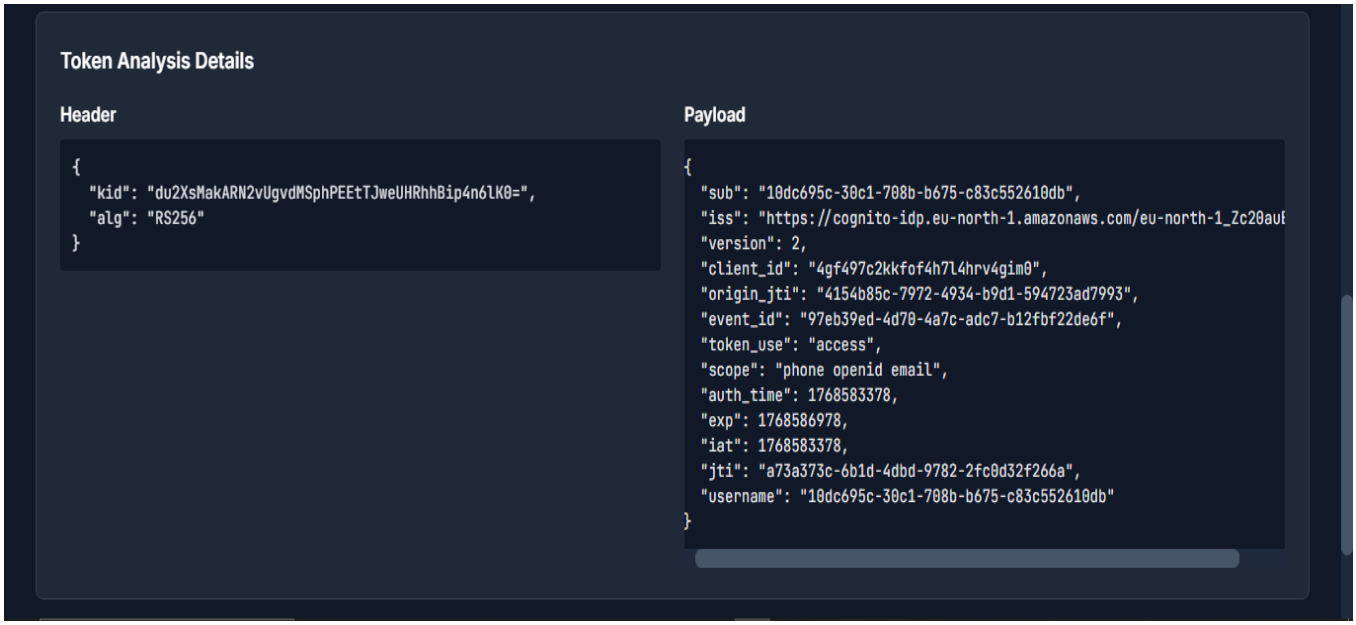
The authentication page opened and I signed in using the previously created test user.



I got an access token in Postman.



I inspected the token on xjwt.io and it is a secure jwt



Token Analysis Details

Header

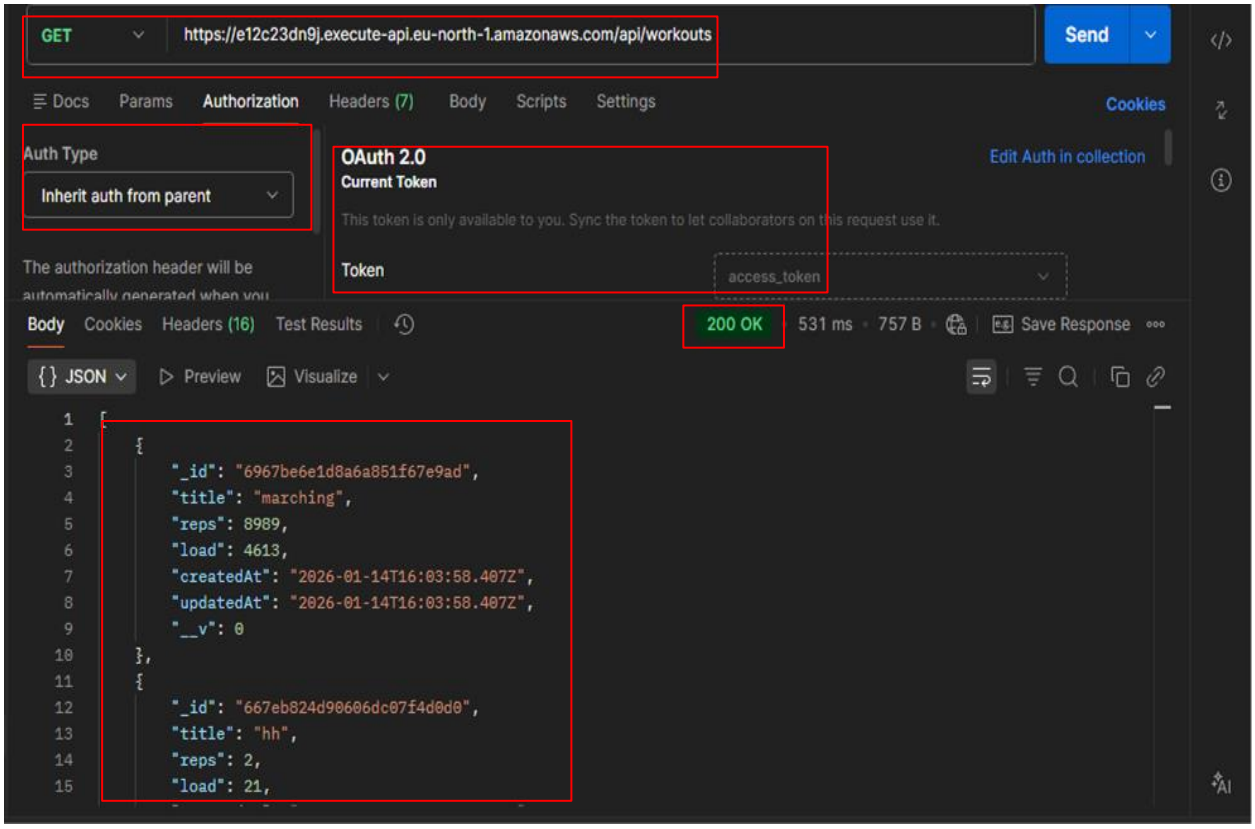

```
{
  "kid": "du2XsMakARN2vUgvdMSphPEEtTJweUHRhhBip4n6lK0=",
  "alg": "RS256"
}
```


Payload


```
{
  "sub": "10dc695c-30c1-708b-b675-c83c552610db",
  "iss": "https://cognito-idp.eu-north-1.amazonaws.com/eu-north-1_Zc20auE
  "version": 2,
  "client_id": "4gf497c2kkfof4h7l4hrv4gim0",
  "origin_jti": "4154b85c-7972-4934-b9d1-594723ad7993",
  "event_id": "97eb39ed-4d70-4a7c-adc7-b12fbf22de6f",
  "token_use": "access",
  "scope": "phone openid email",
  "auth_time": 1768583378,
  "exp": 1768586978,
  "iat": 1768583378,
  "jti": "a73a373c-6b1d-4dbd-9782-2fc0d32f266a",
  "username": "10dc695c-30c1-708b-b675-c83c552610db"
}
```


Now for all my requests I inherited the Auth Type that uses the access token from the parent.

I tested by sending a GET request to API ***/*** endpoint as well as to the **/api/workouts** endpoint.
I successfully accessed the API and this shows that **the API Gateway is secured by OAuth2** since I was now authenticated.



4.2 Rate Limiting:

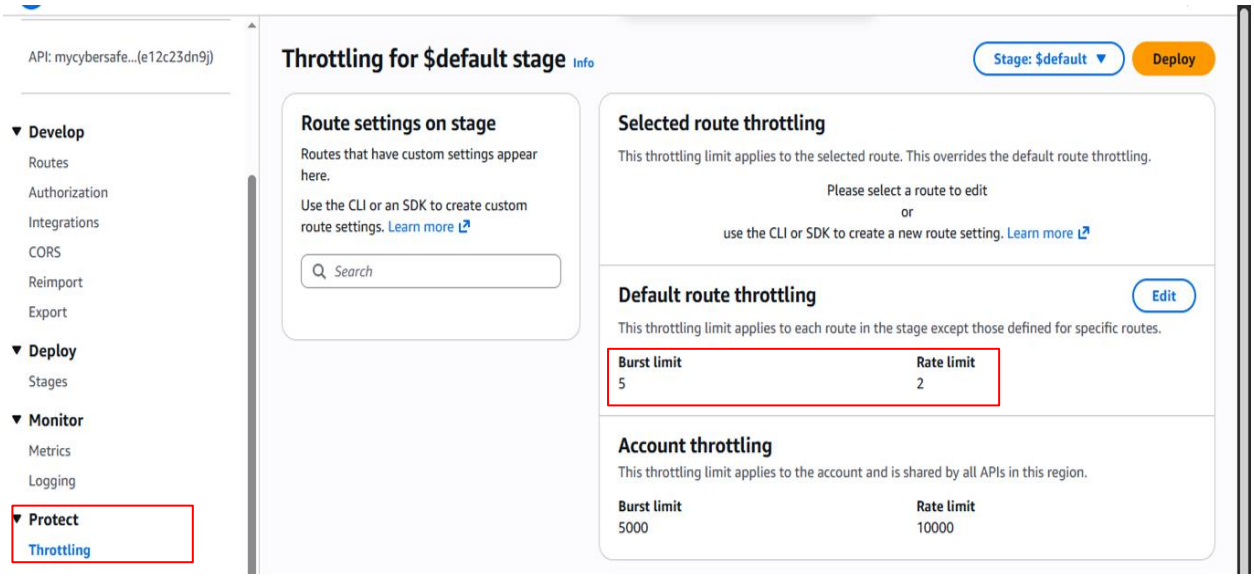
Description

Rate Limiting prevents any single user or a malicious bot from overwhelming an API with too many requests in a short period.

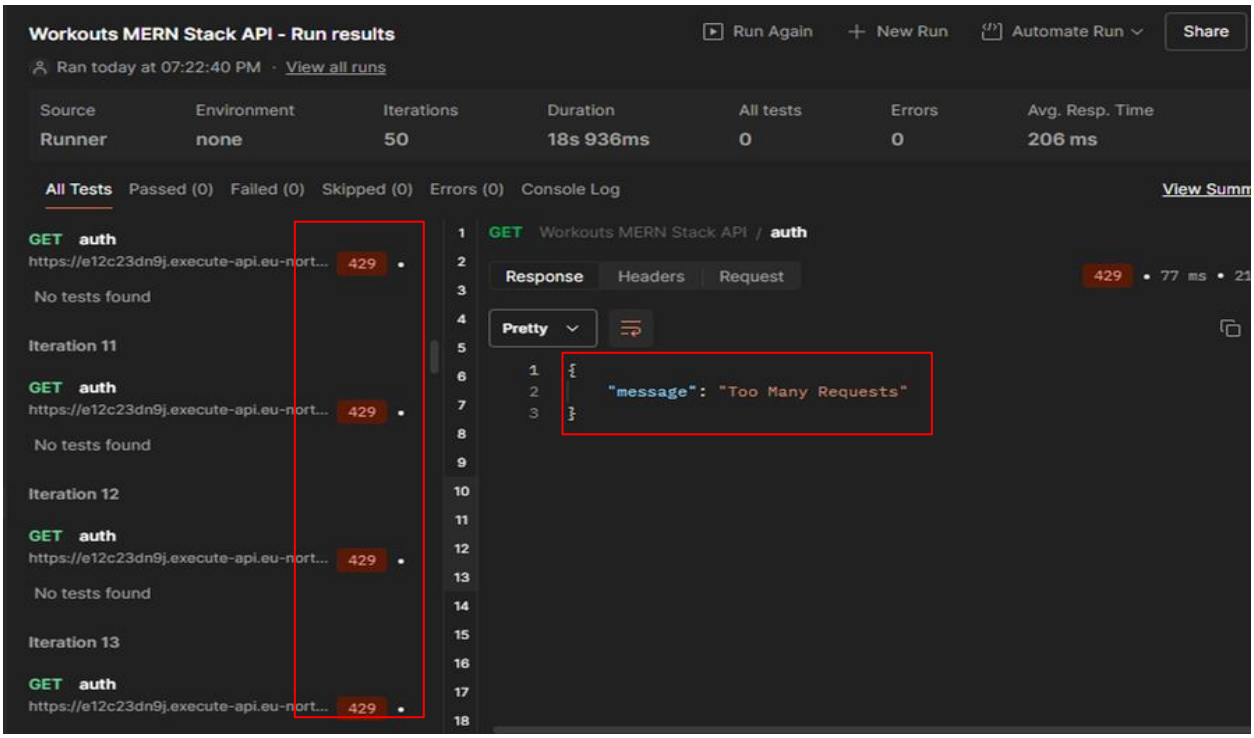
- **How it works in AWS:**
  - **Rate:** The average number of requests allowed per second (example 10 requests/sec).
  - **Burst:** The maximum number of requests a user can make in a sudden spike (example 20 requests).
- **Why it matters:** It prevents **Denial of Service (DoS)** attacks. Even if a hacker has a valid login, they cannot script a loop to crash the Render backend or inflate the database costs by sending millions of requests.
- **Response:** If a user exceeds the limit, the Gateway returns a **429 Too Many Requests** error.

Implementation

- Under the API Gateway, I went to Protect then Selected Throttling.
- I enabled Throttling by setting a **Rate Limit** of 2 requests per second and **Burst Limit** of 5 under Default route throttling.



In Postman Collection I executed 50 iterations and the response status code for some of the requests was 429 - too many requests. This shows that rate limiting was well implemented



4.3 HTTPS-Only

No action needed, AWS API Gateway endpoints are HTTPS by default. HTTP requests are automatically rejected and TLS certificates managed via **AWS ACM**

4.4 Access Control

Description

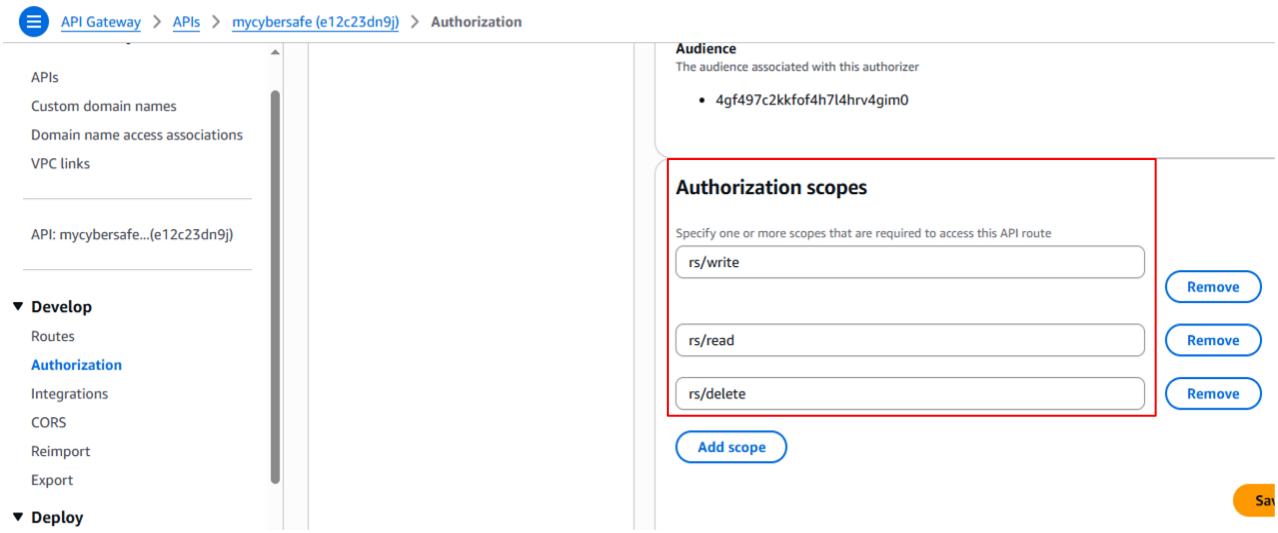
Access Control is the process of verifying what a user is allowed to do and which specific data they are allowed to see.

To implement access control, Amazon Cognito issues JWTs that have default scopes like email, open id etc. To further control access, one can add custom scopes.

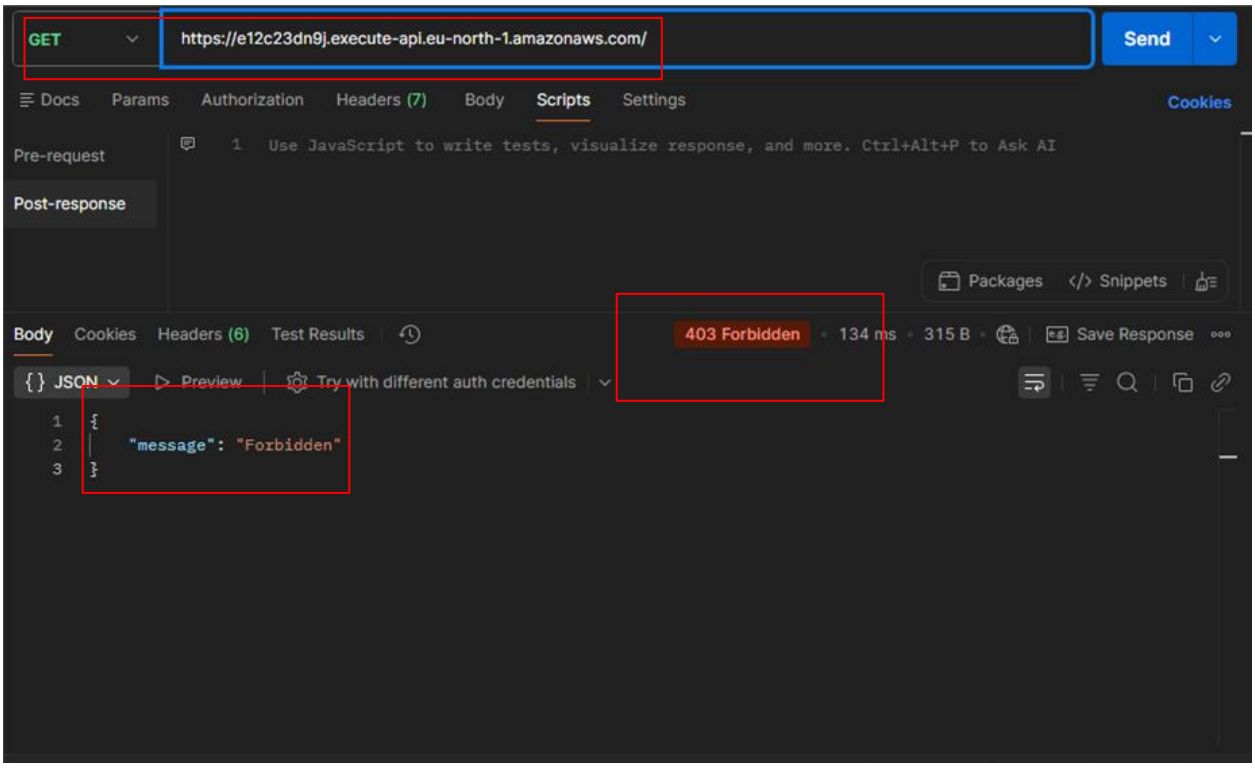
I added custom scopes that are required to access the API, namely read, write and delete. If an access token's claims does not contain any of these scopes, the server must forbid access.

Implementation

I added authorization scopes to the API Gateway

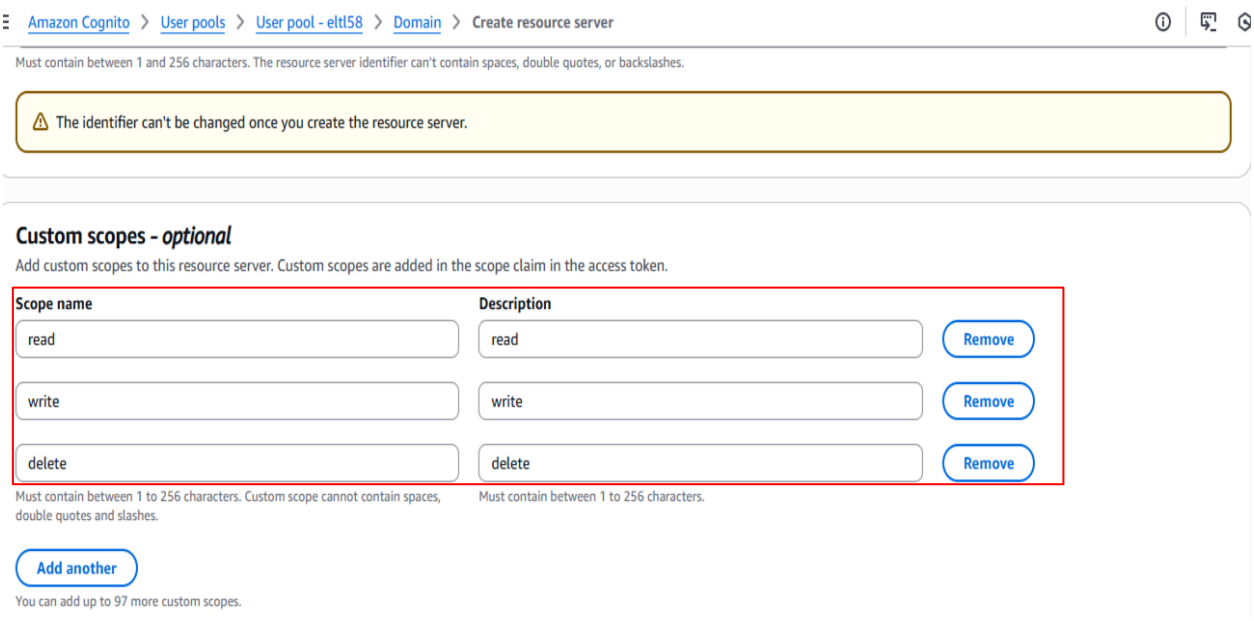


In Postman, I generated a new Access Token without specified scopes and access was forbidden. The user was authenticated but was not allowed to perform the GET request.

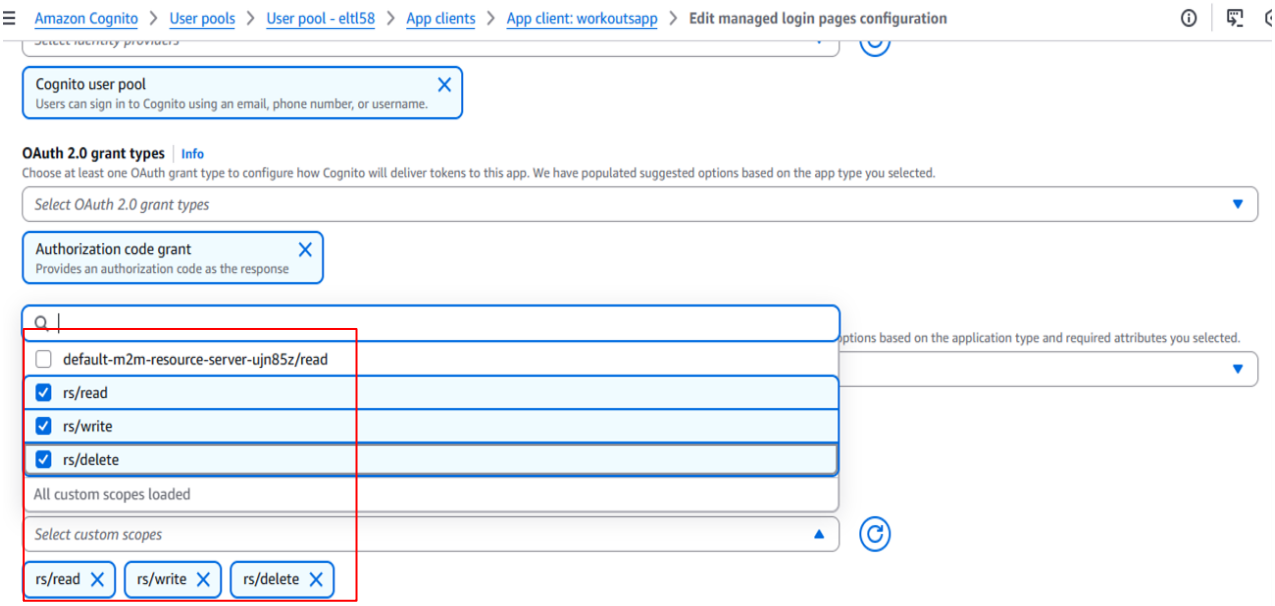


Remediation

In Amazon Cognito, I created a Resource Server under the previously created User pool and set up custom scopes so that the Access Tokens generated by the server will have the required scopes in their claims.



I then attached these custom scopes to the auth app (app client)



After setting up I generated a new Access Token by authenticating the user again and the new token had all the required scopes

DECODED PAYLOAD

JSON CLAIMS TABLE COPY


```
{
  "sub": "10dc695c-30c1-708b-b675-c83c552610db",
  "iss": "https://cognito-idp.eu-north-1.amazonaws.com/eu-north-1_
Zc20auE6U",
  "version": 2,
  "client_id": "4gf497c2kkfof4h7l4hrv4gim0",
  "origin_jti": "fa2653f1-857b-467d-81dd-f4d1cc6d37aa",
  "token_use": "access",
  "scope": "rs/delete phone openid rs/write rs/read email",
  "auth_time": 1768780128,
  "exp": 1768783775,
  "iat": 1768780175,
  "jti": "2a118e2f-ce2d-411a-af8a-179e69eb4aed",
  "username": "10dc695c-30c1-708b-b675-c83c552610db"
}
```


I sent a GET request using the new Access Token and the user was now authorized

GET https://e12c23dn9j.execute-api.eu-north-1.amazonaws.com/ Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Pre-request

Post-response

1 Use JavaScript to write tests, visualize response, and more. Ctrl+Alt+P to Ask AI

Packages Snippets

Body Cookies Headers (16) Test Results 200 OK 480 ms 1.16 KB Save Response

HTML Preview Visualize

Welcome to the Workouts MERN Stack API!

Developed by Charmaine S Mangorima

We collect a large amount of data from our customers with their consent, see [illegible] for contact details

Github: charmainemimic

Conclusion

In this project, a secure API Gateway was successfully implemented using **AWS API Gateway** and connected to a simple backend API. Strong authentication was enforced using **JWT via Amazon Cognito**, ensuring that only authorized users with valid tokens and appropriate scopes could access the API. Additional security measures, including **HTTPS-only access, rate limiting, scope-based access control**, were applied to protect the API from unauthorized access, abuse and common vulnerabilities.

The system was tested to confirm that:

- Authenticated requests succeed and return the expected responses
- Requests with missing or invalid tokens are blocked
- Rate limiting works as intended
- Endpoints are protected based on scopes

Although I did not implement it in the API, to mitigate other authorization vulnerabilities like BOLA **(Broken Object Level Authorization)**, the **sub claim** in the JWT can be used as a tamper-proof identity pass that the client cannot modify. By mapping this verified claim to a backend header via API Gateway, the application ensures that database queries match both the resource ID and the requester's unique identity. This effectively restricts data access to the record owner and neutralizes horizontal privilege escalation.

Overall, the implementation demonstrates a secure API Gateway configuration that follows best practices for authentication, authorization and security hardening.

End of report